

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN - ĐHQG.TPHCM
KHOA KỸ THUẬT MÁY TÍNH
MÔN: KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH
LẬP TRÌNH ASM MIPS

Môi trường thực hành: IDE MARS 4.5

Giảng viên hướng dẫn:	Vũ Đức Lung
Sinh viên thực hiện:	Huỳnh Phạm Ngọc Thắng
Mã số sinh viên (MSSV):	25521672
Lớp học phần:	IT006.R11.VMTN

TPHCM, 10/2026

BÀI TẬP 1

1.1. Nội dung bài toán

Viết chương trình ASM trên IDE Mars 4.5 để nhập từ bàn phím 2 số nguyên, in ra cửa sổ I/O của MARS theo từng yêu cầu sau:

- a. Số lớn hơn
- b. Tổng, hiệu, tích và thương của hai số

1.2. Code Assembly MIPS

```
.data
prompt1: .asciiz "Nhap so thu nhat: "
prompt2: .asciiz "Nhap so thu hai: "
msg_max: .asciiz "a) So lon hon la: "
msg_sum: .asciiz "\nb) Tong = "
msg_diff: .asciiz ", Hieu = "
msg_prod: .asciiz ", Tich = "
msg_quot: .asciiz ", Thuong = "
msg_div_zero: .asciiz ", Thuong: Khong the chia cho 0!"

.text
.globl main

main:
    # -----
    # 1. Nhap hai so nguyen tu ban phim
    # -----
    # In thông báo nhập số thứ nhất
    li $v0, 4
    la $a0, prompt1
    syscall

    # Đọc số thứ nhất (lưu vào $t0)
    li $v0, 5
    syscall
    move $t0, $v0
```

```

# In thong bao nhap so thu hai
li $v0, 4
la $a0, prompt2
syscall

# Doc so thu hai (luu vao $t1)
li $v0, 5
syscall
move $t1, $v0

# -----
# a) Tim va in so lon hon
# -----
li $v0, 4
la $a0, msg_max
syscall

# So sanh $t0 va $t1: neu $t0 >= $t1 thi nhay toi a_is_greater
bge $t0, $t1, a_is_greater
move $a0, $t1      # $t1 lon hon
j print_max

a_is_greater:
    move $a0, $t0      # $t0 lon hon

print_max:
    li $v0, 1          # In gia tri so lon hon
    syscall

# -----
# b) Tinh va in Tong, Hieu, Tich, Thuong
# -----

# --- TONG ($t0 + $t1) ---
li $v0, 4
la $a0, msg_sum
syscall

add $a0, $t0, $t1
li $v0, 1

```

```
syscall
```

```
# --- HIEU ($t0 - $t1) ---
```

```
li $v0, 4
```

```
la $a0, msg_diff
```

```
syscall
```

```
sub $a0, $t0, $t1
```

```
li $v0, 1
```

```
syscall
```

```
# --- TICH ($t0 * $t1) ---
```

```
li $v0, 4
```

```
la $a0, msg_prod
```

```
syscall
```

```
mul $a0, $t0, $t1
```

```
li $v0, 1
```

```
syscall
```

```
# --- THUONG SO THUC ($t0 / $t1) ---
```

```
beqz $t1, divide_by_zero    # Kiem tra truong hop chia cho 0
```

```
# In nhan " Thuong = "
```

```
li $v0, 4
```

```
la $a0, msg_quot
```

```
syscall
```

```
# Step 1: Chuyen du lieu sang thanh ghi FPU ($f0, $f1)
```

```
mtc1 $t0, $f0
```

```
mtc1 $t1, $f1
```

```
# Step 2: Doi dinh dang sang so thuc (Single float)
```

```
cvt.s.w $f0, $f0    # $f0 = (float)$t0
```

```
cvt.s.w $f1, $f1    # $f1 = (float)$t1
```

```
# Step 3: Thuc hien phep chia so thuc
```

```
div.s $f12, $f0, $f1
```

```
# Step 4: In so thuc ra man hinh (Syscall 2)
```

```
li $v0, 2
```

```
syscall
```

```
j end_program
```

```
divide_by_zero:
```

```
li $v0, 4
```

```
la $a0, msg_div_zero
```

```
syscall
```

```
end_program:
```

```
# Ket thuc chuong trinh (Syscall 10)
```

```
li $v0, 10
```

```
syscall
```

1.3. Kết quả thực thi

The screenshot displays the MARS MIPS simulator interface. The main window shows the assembly code for a program that prompts the user for two numbers and performs various calculations. The code is as follows:

```
1 .data
2   prompt1: .asciiz "Nhap so thu nhât: "
3   prompt2: .asciiz "Nhap so thu hai: "
4   msg_max: .asciiz "a) So lon hon la: "
5   msg_sum: .asciiz "(nh) Tong = "
6   msg_diff: .asciiz ", Hieu = "
7   msg_prod: .asciiz ", Tich = "
8   msg_quot: .asciiz ", Thuong = "
9   msg_div_zero: .asciiz ", Thuong: Khong the chia cho 0!"
10
11 .text
12 .globl main
13
14 main:
15   # 1. Nhap hai so nguyen tu ban phim
16   # -----
17   # In thong bao nhap so thu nhât
18   li $v0, 4
19   la $a0, prompt1
20   syscall
21
22   # Doc so thu nhât (luu vao $t0)
23   li $v0, 5
24   syscall
25
```

The bottom panel shows the execution results, including the input values and the calculated results:

```
Mars Messages Run I/O
Nhap so thu nhât: 5
Nhap so thu hai: 10
a) So lon hon la: 10
b) Tong = 15, Hieu = -5, Tich = 50, Thuong = 0.5
-- program is finished running --
```

The right panel displays the registers and condition flags. The registers are organized into three columns: Name, Float, and Double. The condition flags are also shown at the bottom right.

BÀI TẬP 2

2.1. Nội dung bài toán

Viết chương trình ASM trên IDE Mars 4.5 để nhập từ bàn phím một mảng các số nguyên n phần tử (nhập vào số phần tử và giá trị của từng phần tử), xuất ra cửa sổ I/O của MARS theo từng yêu cầu sau:

- Xuất ra giá trị lớn nhất và nhỏ nhất của mảng
- Tổng tất cả các phần tử của mảng

Người sử dụng nhập vào chỉ số của một phần tử nào đó và giá trị của phần tử đó được in ra cửa sổ

2.2. Code Assembly MIPS

```
.data
array:          .space 400          # Cấp phát bộ nhớ cho mảng tối đa 100
phần tử (100 * 4 bytes)
prompt_n:       .asciiz "Nhap so luong phan tu n (1 -> 100): "
prompt_elem1:   .asciiz "Nhap array["
prompt_elem2:   .asciiz "]: "
msg_max:        .asciiz "\na) Gia tri lon nhat: "
msg_min:        .asciiz "\n  Gia tri nho nhat: "
msg_sum:        .asciiz "\nb) Tong cac phan tu: "
prompt_idx1:    .asciiz "\nc) Nhap chi so k phan tu can xem (0 -> "
prompt_idx2:    .asciiz "): "
msg_val1:       .asciiz "  Gia tri array["
msg_val2:       .asciiz "]" = "
msg_invalid:    .asciiz "  Loi: Chi so k nam ngoai pham vi cua mang!"

.text
.globl main

main:
    # -----
    # 1. NHẬP SỐ LƯỢNG PHẦN TỬ (n) VÀ CÁC PHẦN TỬ CỦA MẢNG
    # -----
    # In thông báo nhập n
    li $v0, 4
    la $a0, prompt_n
    syscall
```

```

# Đọc n từ bàn phím (lưu vào $s0)
li $v0, 5
syscall
move $s0, $v0      # $s0 = n

# Khởi tạo biến đếm vòng lặp nhập
li $t0, 0           # $t0 = i = 0
la $s1, array       # $s1 = địa chỉ gốc của mảng (array[0])

input_loop:
    bge $t0, $s0, process_array    # Nếu i >= n thì dừng nhập, chuyển sang xử lý

    # In thông báo "Nhap array[i]: "
    li $v0, 4
    la $a0, prompt_elem1
    syscall

    li $v0, 1
    move $a0, $t0      # In chỉ số i
    syscall

    li $v0, 4
    la $a0, prompt_elem2
    syscall

    # Đọc giá trị nhập vào
    li $v0, 5
    syscall              # Kết quả nhập nằm ở $v0

    # Tính địa chỉ phần tử array[i]: địa chỉ = $s1 + (i * 4)
    sll $t1, $t0, 2     # $t1 = i * 4 (dịch trái 2 bit tương đương nhân 4)
    add $t2, $s1, $t1   # $t2 = địa chỉ thực của array[i]
    sw $v0, 0($t2)      # Lưu giá trị vừa nhập vào array[i]

    addi $t0, $t0, 1    # i++
    j input_loop

# -----
# 2. XỬ LÝ: TÌM MIN, MAX VÀ TÍNH TỔNG (CÂU a & b)
# -----

process_array:
    lw $t3, 0($s1)      # $t3 = min = array[0]
    lw $t4, 0($s1)      # $t4 = max = array[0]
    li $t5, 0           # $t5 = sum = 0
    li $t0, 0           # $t0 = i = 0

```

```

calc_loop:
    bge $t0, $s0, print_results    # Nếu i >= n thì chuyển sang in kết quả

    # Lấy giá trị array[i]
    sll $t1, $t0, 2                # $t1 = i * 4
    add $t2, $s1, $t1              # $t2 = địa chỉ array[i]
    lw $t6, 0($t2)                 # $t6 = array[i]

    # Tính tổng: sum += array[i]
    add $t5, $t5, $t6

    # Cập nhật Min: if (array[i] < min) min = array[i]
    bge $t6, $t3, check_max
    move $t3, $t6

check_max:
    # Cập nhật Max: if (array[i] > max) max = array[i]
    ble $t6, $t4, next_item
    move $t4, $t6

next_item:
    addi $t0, $t0, 1              # i++
    j calc_loop

# -----
# 3. XUẤT KẾT QUẢ CÂU a VÀ CÂU b
# -----

print_results:
    # --- Câu a) In Max ---
    li $v0, 4
    la $a0, msg_max
    syscall

    li $v0, 1
    move $a0, $t4                 # In Max ($t4)
    syscall

    # --- Câu a) In Min ---
    li $v0, 4
    la $a0, msg_min
    syscall

    li $v0, 1
    move $a0, $t3                 # In Min ($t3)
    syscall

```

```
# --- Câu b) In Tổng ---
```

```
li $v0, 4
```

```
la $a0, msg_sum
```

```
syscall
```

```
li $v0, 1
```

```
move $a0, $t5      # In Sum ($t5)
```

```
syscall
```

```
# -----
```

```
# 4. TRUY XUẤT PHẦN TỬ THEO CHỈ SỐ (CÂU c)
```

```
# -----
```

```
# In thông báo "c) Nhập chỉ số k phần tử cần xem (0 -> n-1): "
```

```
li $v0, 4
```

```
la $a0, prompt_idx1
```

```
syscall
```

```
li $v0, 1
```

```
addi $a0, $s0, -1   # In giá trị (n - 1)
```

```
syscall
```

```
li $v0, 4
```

```
la $a0, prompt_idx2
```

```
syscall
```

```
# Đọc chỉ số k
```

```
li $v0, 5
```

```
syscall
```

```
move $t7, $v0      # $t7 = k
```

```
# Kiểm tra điều kiện hợp lệ:  $0 \leq k < n$ 
```

```
bltz $t7, invalid_index    # Nếu  $k < 0$  -> Nhảy tới báo lỗi
```

```
bge $t7, $s0, invalid_index # Nếu  $k \geq n$  -> Nhảy tới báo lỗi
```

```
# Chỉ số hợp lệ: Lấy giá trị array[k]
```

```
sll $t1, $t7, 2      # $t1 =  $k * 4$ 
```

```
add $t2, $s1, $t1     # $t2 = địa chỉ array[k]
```

```
lw $t8, 0($t2)        # $t8 = array[k]
```

```
# In kết quả: "Gia trị array[k] = ..."
```

```
li $v0, 4
```

```
la $a0, msg_val1
```

```
syscall
```

```
li $v0, 1
```

```
move $a0, $t7      # In k
```

```
syscall
```

```
li $v0, 4
```

```
la $a0, msg_val2
```

```
syscall
```

```
li $v0, 1
```

```
move $a0, $t8      # In array[k]
```

```
syscall
```

```
j end_program
```

```
invalid_index:
```

```
li $v0, 4
```

```
la $a0, msg_invalid
```

```
syscall
```

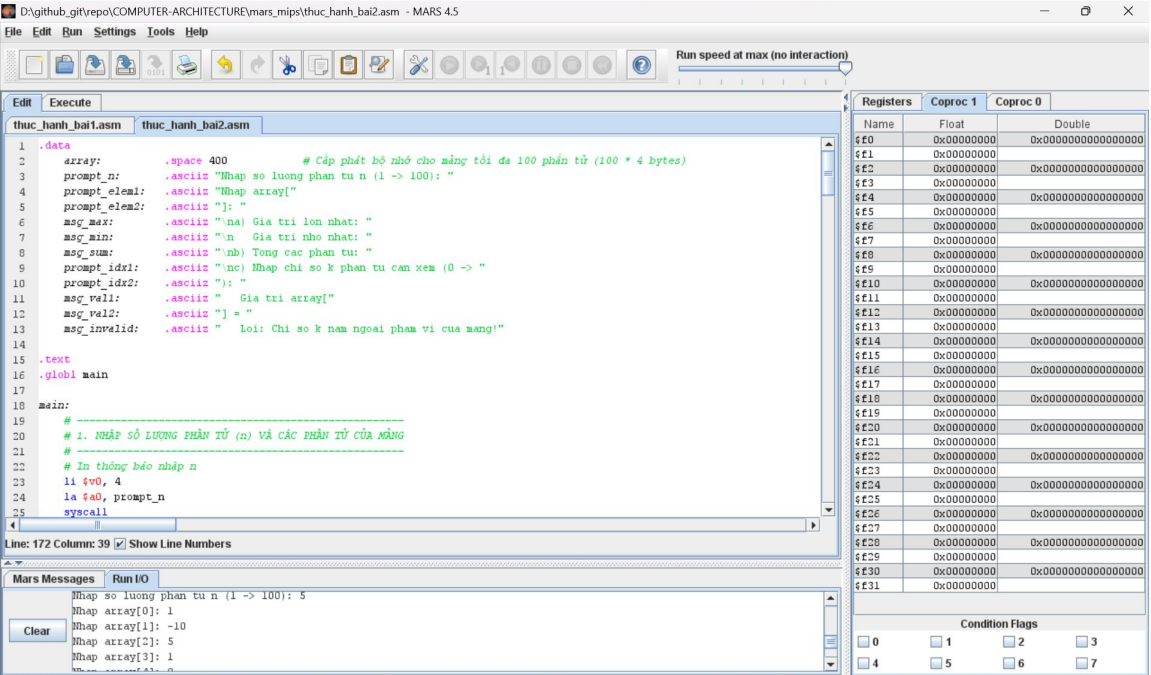
```
end_program:
```

```
# Kết thúc chương trình
```

```
li $v0, 10
```

```
syscall
```

2.3. Kết quả thực thi



The screenshot shows the MARS MIPS simulator interface. The main window displays the assembly code for the program, with line numbers 1 to 25. The code includes data declarations for prompts and messages, and a main function that prompts the user for array size and elements, and then prints the array contents. The 'Registers' window on the right shows the state of registers \$f0 through \$f31, all containing 0x00000000. The 'Mars Messages' window at the bottom shows the output of the program, including prompts for array size and elements, and the resulting array values.

```
1 .data
2     array: .space 400          # Cấp phát bộ nhớ cho mảng tối đa 100 phần tử (100 * 4 bytes)
3     prompt_n: .asciiz "Nhập số lượng phần tử n (1 -> 100): "
4     prompt_elem1: .asciiz "Nhập array["
5     prompt_elem2: .asciiz "]: "
6     msg_max: .asciiz "\na) Giá trị lớn nhất: "
7     msg_min: .asciiz "\n  Giá trị nhỏ nhất: "
8     msg_sum: .asciiz "\nb) Tổng các phần tử: "
9     prompt_idx1: .asciiz "\nc) Nhập chỉ số k phần tử cần xem (0 -> "
10    prompt_idx2: .asciiz "): "
11    msg_val1: .asciiz "  Giá trị array["
12    msg_val2: .asciiz "]= "
13    msg_invalid: .asciiz "  Lỗi: Chỉ số k nằm ngoài phạm vi của mảng!"
14
15 .text
16 .globl main
17
18 main:
19     # --- NHẬP SỐ LƯỢNG PHẦN TỬ (n) VÀ CÁC PHẦN TỬ CỦA MẢNG ---
20     # 1. NHẬP SỐ LƯỢNG PHẦN TỬ (n) VÀ CÁC PHẦN TỬ CỦA MẢNG
21     # In thông báo nhập n
22     li $v0, 4
23     la $a0, prompt_n
24     syscall
25
```

Mars Messages

```
Nhập số lượng phần tử n (1 -> 100): 5
Nhập array[0]: 1
Nhập array[1]: -10
Nhập array[2]: 5
Nhập array[3]: 1
```

Mars Messages

Run I/O

Clear

Nhap so luong phan tu n (1 -> 100): 5
Nhap array[0]: 1
Nhap array[1]: -10
Nhap array[2]: 5
Nhap array[3]: 1
Nhap array[4]: 0

Clear

Nhap array[2]: 5
Nhap array[3]: 1
Nhap array[4]: 0

a) Gia tri lon nhat: 5
Gia tri nho nhat: -10

Clear

Gia tri nho nhat: -10
b) Tong cac phan tu: -3
c) Nhap chi so k phan tu can xem (0 -> 4): 3
Gia tri array[3] = 1
-- program is finished running --